

3. (5) Si consideri il programma sulla destra. Si dica cosa viene calcolato dall'assegnamento in caso di scoping statico (mostrando come vengono calcolati i link statici e come vengono risolti riferimenti non locali) e in caso di scoping dinamico (mostrando l'evoluzione della tabella centrale dei riferimenti (CRT) e come vengono risolti riferimenti non locali).

```
{int a=2;
void pippo(){
  a = a + 2;}
void pluto(){
  int a = 4;
  void minnie(int b){
    int a = b + 4;
    pippo();}
  pippo();
  minnie(4);}
a=3;
pluto(); }
```

Catena di annidamento

Catena di esecuzione

main (a) SD(main)=0
└ pippo SD(pippo)=1
└ pluto (a) SD(pluto)=1
└ minnie (a) SD(minnie)=2

main → pluto → pippo
└ minnie → pippo

Scoping statico

Esecuzione main
a=3 → a locale

Esecuzione main → pluto

$$k = SD(chiamante) - SD(chiamato) + 1$$

$$K_{pluto} = SD(main) - SD(pluto) + 1$$

$$= 0 - 1 + 1 = 0 \Rightarrow CS(pluto) = main$$

Chiamata pluto → pippo

$$K_{pippo} = SD(pluto) - SD(pippo) + 1$$

$$= 1 - 1 + 1 = 1 \Rightarrow CS(pippo) = main$$

Esecuzione pippo: a = a + 2

a non locale

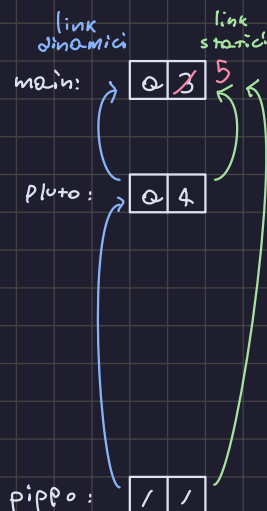
$$N = SD(uso) - SD(def)$$

$$N_a = SD(pippo) - SD(main)$$

$$= 1 - 0 = 1 \Rightarrow a = 3$$

$$a = a + 2 = 3 + 2 = 5$$

Terminazione di pippo
viene rimosso dallo stack



Lo stack diventa

Esecuzione $\text{pluto} \rightarrow \text{minnie}(4)$

$$K_{\text{minnie}} = \text{SD}(\text{pluto}) - \text{SD}(\text{minnie}) + 1$$

$$= 1 - 2 + 1 = 0 \Rightarrow \text{CS}(\text{minnie}) = \text{pluto}$$

Chiamata $\text{minnie} \rightarrow \text{pippo}$

$$K_{\text{pippo}} = \text{SD}(\text{minnie}) - \text{SD}(\text{pippo}) + 1$$

$$= 2 - 1 + 1 = 2 \Rightarrow \text{CS}(\text{pippo}) = \text{main}$$



Esecuzione di: $\text{pippo: } a = a + 2$
 a non locale

$$N_a = \text{SD}(\text{pippo}) - \text{SD}(\text{main})$$

$$= 1 - 0 = 1 \Rightarrow a = 5$$

$$a = a + 2 = 5 + 2 = 7$$

Alla terminazione lo stack viene deallocato

Scoping dinamico

Esecuzione main

$$a \rightarrow \text{main}, 3$$

Chiamata $\text{main} \rightarrow \text{pluto}$

$$a \rightarrow \text{pluto}, 6 \rightarrow \text{main}, 3$$

Esecuzione $\text{pluto} \rightarrow \text{pippo}$

$$a = a + 2 = 4 + 2 = 6$$

Esecuzione $\text{pluto} \rightarrow \text{minnie}(4)$

$$a \rightarrow \text{minnie}, 8 \rightarrow \text{pluto}, 6 \rightarrow \text{main}, 3$$

Esecuzione $\text{minnie} \rightarrow \text{pippo}$

$$a = a + 2 = 8 + 2 = 10$$

Alla terminazione lo stack viene deallocato

4. (6) Spiegare dove e perché nel programma è necessario parlare di regole di scoping, e dove (e perché) anche di regole di binding. Si dica quale risulta essere il valore finale di z nelle varie situazioni, ovvero con: (1) scoping statico; (2) scoping dinamico e deep-binding; (3) shallow-binding (e scoping dinamico).

```
int x = 4; int y = 3;
int pippo (int in){
    return x + in;
}
int f (int h(int b)){
    int x = 6;
    return h(y) + x;
}
{int x = 5; int y = 4;
int z = f(pippo);}
```

{ 1. Ambiente d: definizione (statico)

```
int x = 4;
int y = 3;
```

```
int pippo(int in) {
    return x + in;
}
```

{ 3 Ambiente d: esecuzione (shallow)

```
int x = 6;
return h(y) + x;
}
```

{ 2 Ambiente d: creazione del legame (deep)

```
int x = 5;
int y = 4;
int z = f(pippo);
}
```

Scoping statico Ambiente 1: $x=4$
 $y=3$

$F(pippo) \Rightarrow pippo(y) + x \rightarrow x + in$

Diagram showing static scoping resolution:

- $pippo(y)$ evaluates to 3 (local y).
- x evaluates to 4 (global x).
- in evaluates to 3 (local in).
- $3 + 4 = 7$.
- $7 + 6 = 13$ (where 6 is the global x).
- Final result: 13.

$z = 13$

Scoping dinamico + shallow binding

$F(pippo) \Rightarrow pippo(y) + x \rightarrow x + in$

Diagram showing dynamic scoping with shallow binding:

- $pippo(y)$ evaluates to 4 (local y).
- x evaluates to 6 (local x).
- in evaluates to 4 (local in).
- $4 + 6 = 10$.
- $10 + 6 = 16$ (where 6 is the global x).
- Final result: 16.

$z = 16$

Ambiente 3 $x=6$ (binding)
Ambiente 2 $y=4$ (scoping)

Scoping dinamico + deep binding Ambiente 2: $x=5$
 $y=4$

$F(pippo) \Rightarrow pippo(y) + x \rightarrow x + in$

Diagram showing dynamic scoping with deep binding:

- $pippo(y)$ evaluates to 4 (local y).
- x evaluates to 5 (global x).
- in evaluates to 4 (local in).
- $4 + 5 = 9$.
- $9 + 6 = 15$ (where 6 is the global x).
- Final result: 15.

$z = 15$

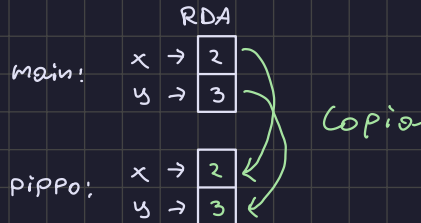
5. (5) Si dica per ognuno dei seguenti metodi di passaggio di parametri, quali sono i valori delle variabili x e y dopo la chiamata di *pippo* nel codice a destra: Passaggio per valore e Passaggio per riferimento. Descrivere dettagliatamente cosa accade nella memoria.

```
void pippo (int a, int b){
    a = 3 * a;
    b = b + 3;
}

void main(){
    int x = 2, y=3;
    pippo(x,y);
}
```

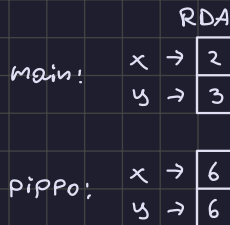
Passaggio per valore

Esecuzione main

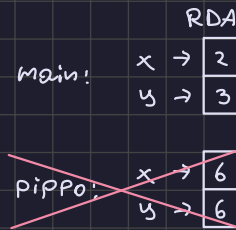


Chiamata pippo

Esecuzione pippo

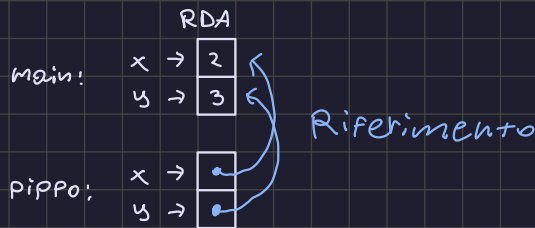


Terminazione pippo



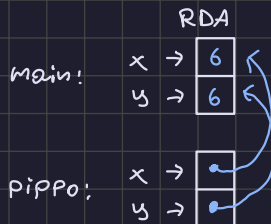
Passaggio per riferimento

Esecuzione main

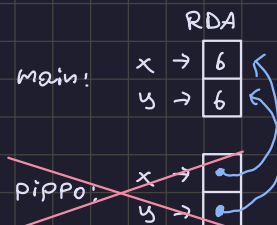


Chiamata pippo

Esecuzione pippo



Terminazione pippo



6. (5) Si descrivano le regole della semantica dinamica per l'assegnamento. Siano $\rho = [x \mapsto l_x, y \mapsto 5]$ un ambiente dinamico con l_x locazione di tipo int, e $\sigma = [l_x \mapsto 3]$ una memoria. Mostrare le derivazioni della semantica dinamica per il comando $x := x * y$ nell'ambiente ρ a partire dalla memoria σ ($\rho \vdash \langle x := x * y, \sigma \rangle \rightarrow \dots$).

$$\rho = [x \mapsto l_x, y \mapsto 5], \sigma = [l_x \mapsto 3]$$

Per valutare $x := x * y$ bisogna prima valutare $x * y$ e quindi si valuta prima x e poi y

Valutazione di x

$$\rho \vdash \langle x, \sigma \rangle \rightarrow \langle 3, \sigma \rangle \quad \rho(\sigma(x)) = 3$$

Valutazione di y

$$\rho \vdash \langle y, \sigma \rangle \rightarrow \langle 5, \sigma \rangle \quad \rho(y) = 5$$

Ora si può valutare $x * y$

Valutazione di $x \cdot y$

$$\frac{\rho \vdash \langle x, \sigma \rangle \rightarrow \langle 3, \sigma \rangle \quad \rho \vdash \langle y, \sigma \rangle \rightarrow \langle 5, \sigma \rangle}{\rho \vdash \langle x \cdot y, \sigma \rangle \rightarrow \langle 3 \cdot 5, \sigma \rangle} \quad \begin{array}{l} \rho(\sigma(x)) = 3 \wedge \\ \rho(y) = 5 \end{array}$$

$$\rho \vdash \langle x \cdot y, \sigma \rangle \rightarrow \langle 15, \sigma \rangle$$

Ora si può valutare la dichiarazione

Valutazione di $x := x \cdot y$

$$\frac{\rho \vdash \langle x \cdot y, \sigma \rangle \rightarrow \langle 15, \sigma \rangle}{\rho \vdash \langle x := x \cdot y, \sigma \rangle \rightarrow \langle x := 15, \sigma \rangle}$$

$$\rho \vdash \langle x := x \cdot y, \sigma \rangle \rightarrow \sigma' = [l_x \mapsto 15]$$